

Documentacion de los Scrapers de Noticias

Explicacion detallada del codigo, medio por medio

1. Conceptos basicos (empezando de cero)

Antes de ver el codigo conviene entender unas pocas ideas. Si nunca has hecho web scraping, este capitulo te da todo lo necesario para seguir el resto del documento. Estan explicadas con analogias y ejemplos; los capitulos siguientes las dan por sabidas.

1.1 Que es el web scraping

'Scraping' (raspar) es programar a la computadora para que ENTRE a paginas web y COPIE datos automaticamente, en vez de hacerlo tu a mano. Imagina un asistente al que le dices: 'abre la seccion de politica de este diario, anota el titulo, la fecha y el enlace de cada noticia, entra a cada una y copiamelo el texto, y guardalo todo en una tabla'. Eso es exactamente lo que hacen estos scripts, pero miles de veces y en segundos.

El resultado final de los seis scrapers es una tabla (un archivo CSV, que se abre en Excel) con una fila por noticia y columnas como titulo, fecha, autor y texto.

1.2 Como se comunican un programa y una pagina web

Cuando abres una pagina en el navegador, por debajo ocurre esto: tu navegador envia una PETICION (request) al servidor del sitio, y el servidor responde con el contenido. Un programa hace lo mismo, solo que sin navegador. La libreria que usamos para eso se llama 'requests', y la peticion mas comun se llama GET (pedir/obtener):

```
import requests
respuesta = requests.get("https://rpp.pe/politica")
print(respuesta.status_code)    # 200
print(respuesta.text)           # el contenido (HTML o JSON) como texto
```

La respuesta trae un 'codigo de estado' (status_code) que dice como fue la peticion. Los que apareceran en el codigo:

- 200 = OK: todo bien, aqui esta el contenido.
- 404 = No encontrado: esa direccion no existe (en Canal N se usa a proposito: si un ID da 404, esa noticia no existe y se salta).
- 403 = Prohibido: el servidor rechaza la peticion (le pasa a La Republica si falta una cabecera; se explica luego).
- 500/502/503/504 = errores temporales del servidor: conviene reintentar (por eso los scripts tienen reintentos automaticos).

Por eso casi todas las funciones revisan 'if status_code != 200' antes de seguir: si no vino un 200, no hay contenido útil que procesar.

1.3 La URL y los parametros de consulta (la 'query')

Una URL (direccion web) tiene partes. Fijate en esta:

```
https://larepublica.pe/api/search/articles?category_slug=politica&page=2&limit=24
```

- https:// -> el protocolo.
- larepublica.pe -> el dominio (el servidor).
- /api/search/articles -> la ruta (que recurso se pide). Cuando una ruta devuelve datos para programas (no una pagina para ver), se le llama 'endpoint' o API.
- ? ... -> aqui empieza la QUERY STRING: una lista de parametros clave=valor separados por '&'. En el ejemplo: category_slug=politica, page=2, limit=24.

La query string es como los filtros de una busqueda: 'quiero la categoria politica, pagina 2, de a 24 resultados'. Cambiando esos valores se pide otra cosa. En el codigo NO se escribe el '?clave=valor' a mano: se arma un diccionario de Python y 'requests' lo convierte en query string automaticamente:

```
params = {"category_slug": "politica", "page": 2, "limit": 24}
requests.get("https://larepublica.pe/api/search/articles", params=params)
# requests construye por ti la URL:
# .../articles?category_slug=politica&page=2&limit=24
```

Ese diccionario 'params' es, literalmente, 'como se construye la query'. En cada diario veras que se arma un diccionario asi (a veces con un numero de pagina que va cambiando en un bucle) para pedir pagina tras pagina.

1.4 Las cabeceras (headers): User-Agent y Referer

Junto con la peticion se envian unas 'cabeceras' (headers): datos extra sobre quien pide. Dos importan aqui:

- User-Agent: identifica que 'navegador' hace la peticion. Muchos sitios ignoran o bloquean a quien no manda uno (asumen que es un bot). Por eso se envia uno de Chrome, para que la peticion parezca de un navegador normal.
- Referer: dice 'desde que pagina vengo'. La API de La Republica EXIGE que el Referer sea su seccion de politica; si falta, responde 403 (prohibido). Es una comprobacion de seguridad del sitio que hay que imitar.

```
HEADERS = {"User-Agent": "Mozilla/5.0 ... Chrome/120 ...",
           "Referer": "https://larepublica.pe/politica"}
requests.get(url, headers=HEADERS)
```

1.5 Dos formas de recibir los datos: JSON y HTML

Un sitio puede entregar la lista de noticias de dos maneras muy distintas, y por eso los scripts se dividen en dos estilos:

(a) Una API que devuelve JSON (datos ya ordenados)

Algunos sitios tienen una 'API': una dirección que, en vez de una página bonita, devuelve los datos crudos en formato JSON. JSON son datos estructurados con dos piezas: OBJETOS entre llaves {clave: valor} (como un diccionario) y LISTAS entre corchetes [...]. Ejemplo:

```
{
  "articles": {
    "data": [
      {"title": "Congreso aprueba ley", "date": "2026-05-11",
       "slug": "/politica/2026/05/11/..."},
      {"title": "Otra noticia", "date": "2026-05-10", "slug": "..."}
    ]
  }
}
```

Trabajar con JSON es cómodo: cada dato ya viene etiquetado. En Python, con 'respuesta.json()' eso se convierte en diccionarios y listas, y se accede así: data['articles']['data'] es la lista de noticias, y de cada una noticia['title'] es el título. El Comercio, El Peruano y La Republica funcionan así.

(b) Una página HTML (hecha para humanos)

Otros sitios no tienen API: solo la página web normal, escrita en HTML. HTML son 'etiquetas' anidadas que forman lo que ves. Una etiqueta tiene nombre, atributos y contenido:

```
<article class="news">
  <h2 class="news__title">
    <a href="/politica/mi-noticia-noticia-12345">Título de la noticia</a>
  </h2>
  <time datetime="2026-05-11">11 de mayo</time>
</article>
```

- 'article', 'h2', 'a', 'time' son las etiquetas (tags).
- class="news" y href="..." son atributos. 'class' e 'id' son etiquetas que el sitio pone para dar estilo; nos sirven para localizar lo que queremos.
- El texto visible ('Título de la noticia') es el contenido.

Aquí hay que 'parsear' (analizar) el HTML para sacar el dato de en medio de todas las etiquetas. Para eso se usa BeautifulSoup, que convierte ese texto en un árbol que se puede buscar. Se localizan elementos con SELECTORES, por ejemplo 'el <a> que esta dentro del <h2> de clase news__title'. Perú21 y RPP funcionan así.

1.6 Las herramientas que se usan

- requests: hace las peticiones a la web (los GET) y trae la respuesta.
- BeautifulSoup (librería bs4): parsea el HTML y permite buscar elementos y sacar su texto.
- json: convierte el texto JSON en diccionarios/listas de Python (para las APIs).
- pandas: maneja la tabla de datos (un 'DataFrame') y la guarda como CSV. Un DataFrame es una tabla en memoria, con filas y columnas, igual que una hoja de Excel.
- re (expresiones regulares): busca patrones en texto, p. ej. sacar el número de ID de una URL o limpiar espacios.

1.7 Como se averigua la API y la estructura de un sitio

Quizá te preguntes: cómo se supo que La República tiene esa API, o que Canal N guarda todo en un JSON escondido. Se descubre con las 'Herramientas de desarrollador' del navegador (tecla F12), en la pestaña 'Network' (Red): ahí se ven TODAS las peticiones que hace la página mientras la usas. Si al hacer clic en 'ver más' aparece una petición a '/api/search/articles?...', ya sabes el endpoint y los parámetros a imitar. Para el HTML, la pestaña 'Elements' muestra las etiquetas y clases de cada parte. Ese trabajo de exploración es previo a escribir el scraper y explica por qué cada diario usa una técnica distinta.

2. Vision general de la arquitectura

Todos los medios comparten el mismo objetivo: recolectar las noticias de politica (o de actualidad, en Canal N) de los ultimos 2 anos y guardarlas en un archivo CSV con un esquema unico de 8 columnas. Lo que cambia entre un medio y otro es UNICAMENTE la forma de obtener la lista de noticias, porque cada sitio web esta construido de manera distinta.

El esquema unico de 8 columnas

Todos los scrapers terminan produciendo un CSV con estas mismas columnas y en este mismo orden. Gracias a eso, los seis archivos se pueden concatenar en un unico conjunto de datos sin conflictos.

medio	# nombre del diario (ej. "El Comercio", "RPP", "Canal N")
fecha	# fecha de publicacion
titulo	# titular de la noticia
autor	# firma (puede quedar vacio si el medio no lo expone)
url	# direccion completa de la noticia
resumen	# bajada / subtítulo / meta-descripcion
tags	# etiquetas separadas por comas
texto_completo	# cuerpo de la noticia en una sola celda

El pipeline de dos etapas

Cinco de los seis medios (todos menos Canal N) usan un pipeline de DOS scripts que se ejecutan en orden:

- Script 01 - Recoleccion de links: recorre el listado/seccion del diario y arma un primer CSV con lo que se puede sacar del listado (como minimo titulo, url y fecha).
- Script 02 - Extraccion de contenido: lee ese CSV, entra a cada noticia una por una, descarga su HTML, limpia el ruido (publicidad, recomendados, tuits) y extrae el cuerpo en la columna texto_completo. Si el listado no traia autor/resumen/tags, tambien los completa aqui.

Se separa en dos etapas porque son tareas distintas: la etapa 01 es rapida (una peticion por pagina de listado) y la etapa 02 es lenta (una peticion por noticia). Separarlas permite revisar el listado antes de lanzar la descarga masiva, y reintentar solo la parte que falle.

Canal N es la excepcion: por como esta hecho su sitio, conviene un unico script que recorre las noticias por su ID y ya trae todos los datos de una vez (se explica en su seccion).

Por que cada medio es diferente

La parte difícil del scraping es siempre la MISMA pregunta: como consigo la lista de todas las noticias de política de los últimos 2 años. Cada sitio la responde distinto:

- El Comercio y La Republica tienen una API interna (JSON) que devuelve la lista paginada. Es lo mas comodo: se piden paginas hasta que las noticias sean muy viejas.
- El Peruano tambien tiene una API, pero de tecnologia ASP.NET, con fechas en un formato especial.
- Peru21 no tiene API: se recorre el HTML de la seccion pagina por pagina.
- RPP tampoco tiene API de listados, pero si un 'archivo' por fecha: se recorre dia por dia hacia atras.
- Canal N no expone ningun listado historico, pero sus noticias tienen un ID numerico correlativo: se recorren los IDs hacia atras.

3. Glosario: conceptos que se repiten en todos

Estas piezas aparecen casi identicas en los seis scrapers. Se explican aqui una vez; en las secciones de cada medio se dan por entendidas y solo se comenta lo particular.

3.1 Anclaje de rutas: BASE_DIR

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
ARCHIVO_SALIDA = os.path.join(BASE_DIR, "..._politica.csv")
```

`__file__` es la ruta del propio script. `abspath` la vuelve absoluta y `dirname` se queda con la carpeta que lo contiene. Asi, los CSV se leen y escriben SIEMPRE en la carpeta del script, sin importar desde donde se ejecute (no depende del 'directorio actual' de la terminal). `os.path.join` une rutas de forma correcta en cualquier sistema operativo.

3.2 Sesión robusta: requests.Session + Retry

```
session = requests.Session()
retries = Retry(
    total=5,                # hasta 5 reintentos
    backoff_factor=1,       # espera creciente: 1s, 2s, 4s...
    status_forcelist=[500,502,503,504], # reintentar ante estos errores
    allowed_methods=["GET"])
session.mount("https://", HTTPAdapter(max_retries=retries))
```

Una Session reutiliza la conexión (mas eficiente que `requests.get` suelto) y guarda configuración comun. El objeto Retry define una política de reintentos: si el servidor responde con un error temporal (500/502/503/504), la petición se reintenta sola hasta 5 veces, esperando cada vez mas (backoff exponencial) para no insistir de golpe. Como un scraper hace miles de peticiones, esto evita que un fallo puntual de red tumbe todo el proceso. `mount` aplica esa política a las URLs http y https.

3.3 Cabeceras: HEADERS y el User-Agent

```
HEADERS = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) ... Chrome/120 ..."
}
```

El User-Agent es el texto con que un navegador se identifica. Muchos sitios rechazan (o responden distinto a) peticiones sin User-Agent, porque asumen que son bots. Al enviar uno de Chrome, la petición parece venir de un navegador real y el servidor responde el HTML normal. En La Republica ademas hace falta la cabecera Referer (se explica en su sección).

3.4 BeautifulSoup: leer y limpiar HTML

BeautifulSoup convierte el HTML (texto) en un arbol de objetos que se puede recorrer. Las operaciones que se repiten en los scripts 02 son:

- `soup = BeautifulSoup(html, 'html.parser')`: construye el arbol a partir del texto.
- `find(...)` / `find_all(...)`: buscan el primer / todos los elementos que cumplen un criterio (por etiqueta, por id, o por clase con una expresion regular).
- `select(...)` / `select_one(...)`: lo mismo pero con selectores CSS (ej. `'article.news'`).
- `decompose()`: ELIMINA un elemento del arbol por completo. Se usa para borrar publicidad, tuits, bloques de 'te recomendamos', etc. antes de leer el texto, para que ese ruido no se cuele en el cuerpo.
- `get_text(separator=' ', strip=True)`: devuelve solo el texto de un elemento. El `separator=' '` evita que se peguen palabras cuando hay etiquetas juntas (negritas, enlaces); `strip=True` quita espacios y saltos sobrantes.

Un patron muy comun es 'localizar el contenedor del cuerpo, hacer `decompose` del ruido, y recien entonces extraer el texto de los parrafos'.

3.5 Corte por antigüedad: ANIOS_MAXIMOS

```
ANIOS_MAXIMOS = 2
# en los que usan API:
def dentro_del_rango(fecha, anios=2):
    dias = (datetime.now() - fecha).days
    return dias <= anios * 365
```

Todos limitan la recoleccion a los ultimos 2 anos. Como los listados vienen ordenados de la noticia mas nueva a la mas vieja, en cuanto aparece una noticia fuera de rango se sabe que todas las siguientes son aun mas viejas: se activa una bandera 'detener' y se corta el bucle. La comparacion se hace en dias (2 anos ~ 730 dias).

3.6 Deduplicar: dict.fromkeys

```
", ".join(dict.fromkeys(lista_de_nombres))
```

`dict.fromkeys` crea un diccionario usando la lista como claves; como las claves de un diccionario no se repiten, esto elimina duplicados PERO conservando el orden original (a diferencia de `set()`, que lo pierde). Se usa para los tags, para no repetir etiquetas.

3.7 Exportar: reindex y to_csv

```
df = df.reindex(columns=["medio", "fecha", "titulo", "autor", "url", "resumen", "tags"])
df.to_csv(ARCHIVO_SALIDA, index=False, encoding="utf-8-sig")
```

`reindex(columns=[...])` fuerza el orden EXACTO de columnas del esquema comun (y descarta cualquier columna extra). `to_csv` escribe el DataFrame a disco: `index=False` evita agregar la columna de indice de pandas, y `encoding='utf-8-sig'` guarda en UTF-8 con BOM, que es lo que hace que Excel abra bien las tildes y la n.

3.8 Patron reanudable (etapas 02 y Canal N)

Los scripts largos (los 02 y Canal N) guardan el progreso cada cierto numero de noticias y, al re-ejecutarse, continuan donde quedaron en vez de empezar de cero. Hay dos variantes: releer el CSV de salida y saltar las filas que ya tienen texto (RPP, Peru21, El Peruano), o guardar el ultimo ID en un archivo de checkpoint (Canal N). Es importante porque una corrida completa puede tardar horas y podria cortarse.

3.9 Fechas epoch: fromtimestamp

```
datetime.fromtimestamp(int(pubtime), tz=ttimezone.utc).strftime("%Y-%m-%d")
```

Algunos sitios entregan la fecha como 'epoch': un numero de segundos (o milisegundos) desde el 1 de enero de 1970. `fromtimestamp` lo convierte en una fecha real, y `strftime` la formatea como texto 'AAAA-MM-DD'. Aparece en El Peruano (milisegundos) y Canal N (segundos).

El script solo tiene que leer cada uno de esos campos y copiarlo a su columna. Ese 'mapeo campo -> columna' es toda la logica de la etapa 01.

4.2 Script 01 - recoleccion de links (API Arc)

Configuracion

```
BASE_URL = "https://elcomercio.pe/pf/api/v3/content/fetch/story-feed-by-section"
STORIES_QTY = 100 # noticias por pagina
INCLUDED_FIELDS =
("headlines.basic,subheadlines.basic,credits.by._id,credits.by.name,\n
 websites.elcomercio.website_url,content_restrictions.content_code,\n
 display_date,taxonomy.tags.text")
CODIGOS_DE_PAGA = {"metered", "premium"}
EXCLUIR_NOTAS_DE_PAGA = True
```

- BASE_URL es el endpoint de la API de Arc para 'feed por seccion'.
- STORIES_QTY = 100: se piden 100 noticias por llamada. El desplazamiento (offset) avanza de 100 en 100.
- INCLUDED_FIELDS le dice a la API que SOLO devuelva esos campos. Se reduce a lo que se usa para que la respuesta pese menos y viaje mas rapido.
- CODIGOS_DE_PAGA es un conjunto con los codigos que marcan una nota de pago ('metered' = medidor/paywall). EXCLUIR_NOTAS_DE_PAGA=True hace que esas notas se descarten (no tendrian cuerpo accesible en la etapa 02).

Funcion dentro_del_rango: fechas en formato ISO

```
def dentro_del_rango(fecha_str, anios=2):
    fecha_str = fecha_str.replace("Z", "+00:00")
    fecha = datetime.fromisoformat(fecha_str)
    if fecha.tzinfo is None:
        fecha = fecha.replace(tzinfo=timezone.utc)
    dias = (datetime.now(timezone.utc) - fecha).days
    return dias <= anios * 365
```

La API entrega la fecha en formato ISO 8601 (ej. '2026-05-11T17:25:59.123Z'). La 'Z' final significa 'zona horaria UTC', pero fromisoformat no la entiende, así que se reemplaza por '+00:00', que es lo mismo escrito de una forma que si acepta. Si por algun motivo la fecha no trae zona horaria, se le asigna UTC. Luego se calcula la diferencia en dias contra 'ahora' (tambien en UTC, para comparar peras con peras) y se verifica el rango.

Funciones construir_query y obtener_noticias_pagina

```
def construir_query(feed_offset):
    query = {"section": "/politica", "feedOffset": feed_offset,
            "stories_qty": STORIES_QTY, "includedFields": INCLUDED_FIELDS}
```

```
return json.dumps(query, separators=(",", ":"))
```

La API espera un parametro llamado 'query' que es, el mismo, un JSON. `construir_query` arma ese diccionario (seccion politica, desde que offset, cuantas noticias y que campos) y `json.dumps` lo convierte a texto. `separators=(',', ':')` lo escribe sin espacios, compacto, tal como lo manda el navegador.

```
def obtener_noticias_pagina(feed_offset):
    params = {"query": construir_query(feed_offset), "_website": "elcomercio"}
    response = requests.get(BASE_URL, params=params, timeout=30, headers={...})
    if response.status_code != 200:
        return None
    return response.json()
```

Hace la peticion GET con ese query y con `_website='elcomercio'`. Si el servidor no responde 200 (exito), devuelve None (que el bucle interpreta como 'ya no hay mas o hubo error'). Si todo va bien, `response.json()` convierte la respuesta en un diccionario de Python.

Bucle de recoleccion

```
feed_offset = 0
while True:
    data = obtener_noticias_pagina(feed_offset)
    if data is None: break
    articulos = data.get("content_elements", [])
    if not articulos: break
    for articulo in articulos:
        fecha = articulo.get("display_date")
        if not dentro_del_rango(fecha, ANIOS_MAXIMOS):
            detener = True; continue
        ... extraer autor, titulo, resumen, tags, url, es_paga ...
    if detener: break
    feed_offset = data.get("next") # la API dice cual es el siguiente offset
```

El bucle infinito 'while True' se corta desde adentro. En cada vuelta pide una pagina de la API; `content_elements` es la lista de noticias. Si viene vacia, se acabo el listado. Por cada articulo revisa la fecha: si esta fuera de rango marca 'detener'. La clave 'next' que devuelve la API indica el offset de la pagina siguiente; cuando no viene, se llego al final.

Extraccion de campos de cada articulo

```
autor = ""
creditos = articulo.get("credits", {}).get("by", [])
if creditos: autor = creditos[0].get("name", "")
titulo = articulo.get("headlines", {}).get("basic", "")
resumen = articulo.get("subheadlines", {}).get("basic", "")
taxonomia = articulo.get("taxonomy", {}) or {}
tags = ", ".join(dict.fromkeys(
    t["text"] for t in (taxonomia.get("tags") or []) if t.get("text")))
url_relativa = articulo.get("websites", {}).get("elcomercio", {}).get("website_url", "")
```

```
url_completa = "https://elcomercio.pe" + url_relativa if url_relativa.startswith("/")
else url_relativa
content_code = (articulo.get("content_restrictions", {}) or {}).get("content_code", "")
es_paga = content_code in CODIGOS_DE_PAGA
if es_paga and EXCLUIR_NOTAS_DE_PAGA: continue
```

Cada `.get('clave', valor_por_defecto)` accede al JSON sin reventar si la clave no existe. El autor esta en `credits.by` (una lista); se toma el primero. Los tags estan en `taxonomy.tags` (cada uno con 'text') y se unen deduplicados. La url viene relativa ('/politica/...') y se le antepone el dominio. Si la nota es de pago y se decidio excluirlas, 'continue' la salta. Las notas validas se agregan a la lista 'noticias' como diccionarios del esquema comun.

4.3 Script 02 - extraccion de contenido

Este script lee `elcomercio_politica.csv`, entra a cada url y saca el cuerpo. Su corazon es la funcion `extraer_contenido_noticia`.

Localizar el contenedor del cuerpo

```
parrafo_muestra = soup.find("p", attrs={"itemprop": "description"})
if parrafo_muestra:
    contenedor_principal = parrafo_muestra.find_parent(["div", "article", "section"])
else:
    contenedor_principal = None
if not contenedor_principal:
    contenedor_principal = soup.find("article")
if not contenedor_principal:
    return ""
```

El Comercio marca los parrafos del cuerpo con `itemprop='description'`. La tecnica es: encontrar UN parrafo del cuerpo y luego subir a su elemento padre (`find_parent`), que es el contenedor que agrupa a todos. Si no se encuentra, se cae a `<article>`. Si tampoco, se devuelve texto vacio.

Fase de decompose (quitar ruido)

```
for embed in contenedor.find_all(["blockquote", "iframe", "script", "style", "figure"]):
    embed.decompose()
for bloque in contenedor.find_all(["ul", "div"],
    class_=re.compile(r"related|interlinking|recommend|tags-list", re.I)):
    bloque.decompose()
for widget in contenedor.find_all(
    class_=re.compile(r"twitter|social|embed|advert|banner", re.I)):
    widget.decompose()
```

Antes de leer el texto se ELIMINAN del contenedor: los embeds (tuits en `blockquote`, videos en `iframe`, figuras, scripts); los bloques de 'te puede interesar' y enlaces relacionados (clases `related/interlinking/recommend/tags-list`); y cualquier widget social/publicitario.

re.compile(..., re.I) busca esas palabras en el nombre de clase sin distinguir mayusculas. Asi el cuerpo queda limpio.

Extraccion del texto

```
elementos = contenedor.find_all(["p","h2","h3","li"], attrs={"itemprop":"description"})
if not elementos:
    elementos = contenedor.find_all(["p","h2","h3","li"])
for el in elementos:
    texto = re.sub(r"\s+", " ", el.get_text(separator=" ", strip=True)).strip()
    if not texto: continue
    if any(f in texto.upper() for f in ["LEE:", "MAS:", "TE PUEDE INTERESAR", ...]):
        continue
    if el.name == "li": texto = f"- {texto}"
    texto_limpio.append(texto)
texto_final = " ".join(texto_limpio)
```

Se toman primero los parrafos marcados como cuerpo (itemprop='description'); si no hubiera, se toman todos los p/h2/h3/li. De cada uno se saca el texto con separador de espacio, se colapsan los espacios multiples, se descartan los vacios y las lineas que contengan frases de recomendacion (filtro de seguridad por si algo escapo al decompose). A los se les pone un guion. Todo se une en una sola cadena: el cuerpo de la noticia.

El bucle principal recorre el CSV, llama esta funcion por cada url, hace una pausa entre peticiones y al final guarda el CSV con la nueva columna texto_completo.

5. El Peruano

El Peruano tiene una API (tecnología ASP.NET) que devuelve la lista de noticias por seccion. La particularidad esta en el formato de la fecha, que viene envuelto como `'/Date(...)/'`.

5.1 Como se construye la peticion (ejemplo real)

Aqui la query es mas directa que en El Comercio: son tres parametros normales (clave=valor). Se arma un diccionario y se le agrega el numero de pagina en cada vuelta del bucle:

```
params = {
    "idsec": 1,          # id de la seccion (1 = Politica)
    "pageSize": 100,     # noticias por pagina
    "pageIndex": 1       # numero de pagina (va cambiando: 1, 2, 3, ...)
```

La URL que resulta:

```
GET https://elperuano.pe/portal/_GetNoticiasSeccionPagingWorker
?idsec=1&pageSize=100&pageIndex=1
```

La respuesta es una LISTA de noticias en JSON (sin envoltorio). Los nombres de los campos vienen de la base de datos de El Peruano (dtm = fecha, vch = texto):

```
[
  {
    "dtmFecha": "/Date(1781758800000)/",  -> fecha (epoch en milisegundos)
    "vchTitulo": "Titulo de la noticia",  -> titulo
    "vchDescripcion": "Bajada / resumen", -> resumen
    "URLFriendly": "noticia-slug-12345"   -> url (se le antepone el dominio)
  },
  ... mas noticias ...
]
```

El `'/Date(1781758800000)'` es la fecha en formato epoch (ver glosario 3.9): 1781758800000 milisegundos desde 1970. Se extrae ese numero con una expresion regular. El Peruano NO manda autor ni tags en la API, por eso esas columnas quedan vacias y se completan en la etapa 02.

5.2 Script 01 - recoleccion de links (API ASP.NET)

Configuracion

```
BASE_URL = "https://elperuano.pe/portal/_GetNoticiasSeccionPagingWorker"
PARAMS_BASE = {
    "idsec": 1,          # ID de la seccion (1 = Politica)
```

```
"pageSize": 100    # noticias por pagina
}
```

idsec=1 selecciona la seccion Politica; pageSize=100 pide 100 por pagina. En cada llamada se agrega el numero de pagina: params['pageIndex'] = page.

Funcion parsear_fecha_asp: el formato /Date(...)/

```
def parsear_fecha_asp(fecha_str):
    if not fecha_str: return datetime.now()
    match = re.search(r"\d+", fecha_str)
    if match:
        timestamp_ms = int(match.group())
        return datetime.fromtimestamp(timestamp_ms / 1000.0)
    return datetime.now()
```

ASP.NET serializa las fechas como '/Date(1781758800000)/', donde el numero son milisegundos epoch. re.search(r'\d+') extrae ese numero (la primera secuencia de digitos). Se divide entre 1000 para pasar de milisegundos a segundos (que es lo que espera fromtimestamp) y se obtiene la fecha. Si no hay fecha, se usa 'ahora' como valor por defecto.

Bucle de recoleccion

```
pagina = 1
while True:
    articulos = obtener_noticias_pagina(pagina)
    if not articulos: break
    for articulo in articulos:
        fecha_obj = parsear_fecha_asp(articulo.get("dtmFecha", ""))
        if not dentro_del_rango(fecha_obj, ANIOS_MAXIMOS):
            detener = True; continue
        url_amigable = articulo.get("URLFriendly", "")
        url_completa = f"https://elperuano.pe/{url_amigable}" if url_amigable else ""
        noticia = {"medio": "El Peruano", "fecha": fecha_obj.strftime("%Y-%m-%d
%H:%M:%S"),
                    "titulo": articulo.get("vchTitulo", "").strip(),
                    "autor": "", "url": url_completa,
                    "resumen": articulo.get("vchDescripcion", "").strip(), "tags": ""}
        if detener: break
    pagina += 1
```

La API devuelve directamente una lista de articulos (sin envoltorio). Los campos tienen nombres propios de la base de datos de El Peruano: dtmFecha (fecha), vchTitulo (titulo), vchDescripcion (resumen), URLFriendly (parte final de la url). El Peruano NO entrega autor ni tags en esta API, por eso quedan en ". El corte por fecha es igual que en los demas.

5.3 Script 02 - extraccion de contenido

Usa la sesion robusta con reintentos (glosario 3.2). La funcion `extraer_contenido_noticia` localiza el cuerpo, limpia y extrae.

Localizar el contenedor y limpiar

```

contenedor = soup.find("div", id="contenido")
if not contenedor:
    contenedor = soup.find("div", class_="nota-contenido")
if not contenedor:
    return ""
# quitar la fecha
for tiempo in contenedor.find_all("time"): tiempo.decompose()
# quitar tuits: por la clase estandar y por las clases internas de Twitter/X
for tuit in contenedor.find_all(["div", "blockquote"],
                                class_=re.compile(r"twitter-tweet", re.I)): tuit.decompose()
for bt in contenedor.find_all(class_=re.compile(r"css-9010ao|r-poi1n3|css-16my406")):
    bt.decompose()

```

El cuerpo esta en `<div id='contenido'>` (con un respaldo por clase `'nota-contenido'`). Se quitan: el `<time>` de la fecha; los tuits incrustados, primero por la clase estandar `'twitter-tweet'` y luego por las clases internas ofuscadas que genera el widget de Twitter/X (`css-9010ao...`), que a veces dejan texto suelto.

Quitar el bloque 'Lea tambien'

```

patron = re.compile(r"(Lea tambien en El Peruano|Tambien lea en El Peruano)", re.I)
for texto_basura in contenedor.find_all(string=patron):
    padre = texto_basura.parent
    if padre:
        contenedor_rel = padre.find_parent(["div", "p", "section"])
        (contenedor_rel or padre).decompose()

```

Aqui se busca por TEXTO (`find_all(string=patron)`) la frase `'Lea tambien en El Peruano'`. Por cada aparicion se sube al elemento que la contiene y se elimina ese bloque entero de enlaces recomendados. Es una limpieza por contenido, util cuando el bloque no tiene una clase CSS facil de apuntar.

Extraccion del texto

```

texto_crudo = contenedor.get_text(separator=" ", strip=True)
texto_final = re.sub(r"\s+", " ", texto_crudo).strip()

```

En El Peruano los parrafos son `<div>`, no `<p>`, asi que en vez de recorrer parrafos uno por uno se toma TODO el texto del contenedor ya limpio de una sola pasada. `separator=' '` evita palabras pegadas y `strip=True` descarta divs vacios. Luego se colapsan espacios. El script 02 es reanudable: relee el CSV de salida y salta las filas que ya tienen texto (glosario 3.8),

guardando cada 20 noticias.

6. La Republica

La Republica tiene una API de busqueda de articulos (JSON) que devuelve la lista por categoria y pagina. El detalle importante: la API responde 403 (prohibido) si no se envia la cabecera Referer, porque comprueba que la peticion 'venga' de su propia web.

6.1 Como se construye la peticion (ejemplo real)

La query son parametros normales, pero con un requisito extra en las cabeceras. Primero los parametros:

```
params = {
  "category_slug": "politica",    # seccion
  "limit": 24,                    # noticias por pagina
  "order_by": "update_date",      # ordenadas de la mas nueva a la mas vieja
  "view": "section",
  "page": 1                       # numero de pagina (va cambiando)
}
```

Y la clave: la cabecera Referer. Sin ella, la API contesta 403 y no da nada. Con ella, la peticion parece venir de la propia seccion de politica del sitio:

```
HEADERS = {"User-Agent": "Mozilla/5.0 ... Chrome/120 ...",
            "Referer": "https://larepublica.pe/politica"} # OBLIGATORIO
GET
https://larepublica.pe/api/search/articles?category_slug=politica&limit=24&order_by=update_date&view=section&page=1
```

La respuesta trae la lista ANIDADA dentro de articles -> data. Cada noticia tiene ademas un sub-objeto 'data' con autor, tags y resumen:

```
{
  "articles": {
    "data": [                                     <- lista de noticias
      {
        "title": "Titulo",                       -> titulo
        "date": "2026-05-11 17:25:59",           -> fecha
        "slug": "/politica/2026/05/11/...",      -> url (se antepone el dominio)
        "data": {
          "teaser": "Bajada / resumen",           -> resumen
          "authors": [{"fullname": "Autor X"}],    -> autor
          "tags": [{"name": "Congreso"}]          -> tags
        }
      }
    ]
  }
}
```

Para leer la lista se navega el JSON: data['articles']['data']. Fijate que autor, tags y resumen

están un nivel más adentro, en artículo['data'].

6.2 Script 01 - recolección de links (API con Referer)

Configuración y cabeceras

```
BASE_URL = "https://larepublica.pe/api/search/articles"
PARAMS_BASE = {"category_slug": "politica", "limit": 24,
               "order_by": "update_date", "view": "section"}
HEADERS = {
    "User-Agent": "Mozilla/5.0 ... Chrome/120 ...",
    "Accept": "application/json, text/plain, */*",
    "Accept-Language": "es-PE, es;q=0.9",
    "Referer": "https://larepublica.pe/politica",
}
```

category_slug='politica' filtra la sección; limit=24 son las noticias por página; order_by='update_date' las ordena de la más reciente a la más antigua (clave para el corte por fecha). En HEADERS, además del User-Agent, el Referer 'https://larepublica.pe/politica' es OBLIGATORIO: simula que la petición proviene de la sección de política; sin él, la API responde 403 y no devuelve nada.

Función dentro_del_rango

```
def dentro_del_rango(fecha_str, años=2):
    fecha = datetime.strptime(fecha_str, "%Y-%m-%d %H:%M:%S")
    dias = (datetime.now() - fecha).days
    return dias <= años * 365
```

Aquí la fecha viene como texto simple '2026-05-11 17:25:59'. strptime la lee con ese formato exacto ('%Y-%m-%d %H:%M:%S') y la convierte en fecha. Es más directo que en El Comercio porque no hay zonas horarias de por medio.

Bucle y extracción de campos (JSON anidado)

```
articulos = data["articles"]["data"]      # la lista está anidada aquí
for articulo in articulos:
    fecha = articulo["date"]
    if not dentro_del_rango(fecha, ANIOS_MAXIMOS):
        detener = True; continue
    autores = articulo["data"].get("authors", [])
    autor = autores[0].get("fullname", "") if autores else ""
    tags = articulo["data"].get("tags", [])
    tags_texto = ", ".join(tag["name"] for tag in tags)
    noticia = {"medio": "La Republica", "fecha": fecha,
               "titulo": articulo["title"], "autor": autor,
               "url": "https://larepublica.pe" + articulo["slug"],
               "resumen": articulo["data"].get("teaser", ""), "tags": tags_texto}
```

La respuesta trae la lista dentro de data['articles']['data']. Cada artículo tiene el título en

'title', la fecha en 'date', la url en 'slug' (relativa, se le antepone el dominio), y un sub-objeto 'data' con el autor (authors[0].fullname), los tags (lista de objetos con 'name') y el resumen (teaser). El corte por fecha y el export al CSV siguen el patron comun.

6.3 Script 02 - extraccion de contenido

Contenedor y decompose

```

contenedor = soup.find("div", id="interna_content")
if not contenedor: contenedor = soup.find("article")
if not contenedor: return ""
# A) bajada/resumen (ya la tenemos en el CSV)
teaser = contenedor.find("h2", class_=re.compile(r"teaser"))
if teaser: teaser.decompose()
# B..E) interlinking, citas "PUEDES VER", modulo de IA, lista de tags
for link in contenedor.find_all("ul", class_=re.compile(r"Interlinking")):
    link.decompose()
for cita in contenedor.find_all("div", class_=re.compile(r"Quote_wrapper")):
    cita.decompose()
for b in contenedor.find_all(class_=re.compile(r"aiContent")): b.decompose()
for t in contenedor.find_all("ul", class_=re.compile(r"tags-list")): t.decompose()

```

El cuerpo esta en <div id='interna_content'> (respaldo: <article>). Antes de extraer se quitan: la bajada (h2 con clase 'teaser', que ya se guardo como resumen en la etapa 01); el bloque de 'interlinking' (noticias incrustadas que enlazan a otras); las citas de recomendacion 'PUEDES VER' (clase Quote_wrapper); el modulo flotante de IA 'Leer resumen' (clase aiContent); y la lista de tags del final. Cada uno se identifica por su clase con una expresion regular.

Extraccion del texto

```

elementos = contenedor.find_all(["p", "h2", "h3", "li"])
for el in elementos:
    texto = re.sub(r"\s+", " ", el.get_text(separator=" ", strip=True)).strip()
    if not texto: continue
    if "PUEDES VER" in texto.upper() or "TE RECOMENDAMOS" in texto.upper() \
        or "LEER RESUMEN" in texto.upper(): continue
    if el.name == "li": texto = f"- {texto}"
    texto_limpio.append(texto)

```

Se recorren parrafos, subtítulos y listas; se normaliza el texto, se descartan vacíos y líneas con frases de recomendación (segundo filtro por si algo escapo al decompose), y se unen. Este 02 procesa el CSV de corrido (no reanudable) y agrega la columna texto_completo.

7. Peru21

Peru21 usa el CMS 'Sirius Publisher' (Drupal) y NO expone una API de listados. En cambio, su seccion de politica se pagina por la ruta: /politica/ (pagina 1), /politica/2/, /politica/3/, etc. Una pagina inexistente devuelve 0 noticias, lo que sirve de corte natural. Aqui SI hay que analizar el HTML del listado en la etapa 01.

7.1 Como se construye la peticion (ejemplo real)

Aqui NO hay query string ni API: la 'consulta' es simplemente la URL de la pagina, y el numero de pagina va en la propia RUTA. La pagina 1 es la ruta base y el resto lleva el numero al final:

```
Pagina 1: GET https://peru21.pe/politica/
Pagina 2: GET https://peru21.pe/politica/2/
Pagina 3: GET https://peru21.pe/politica/3/ ... y asi sucesivamente
```

La respuesta NO es JSON, es la pagina HTML completa. Hay que 'parsearla' con BeautifulSoup para sacar los datos. Cada noticia del listado viene en una tarjeta con esta forma (simplificada):

```
<article class="node--type-article">
  <a href="/politica/mi-noticia/">                                <- la url de la noticia
    <h2 class="titulo-teaser">Titulo</h2>                        <- el titulo
  </a>
  <span class="fecha-teaser">16/07/2026</span>                  <- la fecha
</article>
```

El script localiza cada tarjeta con el selector 'article.node--type-article' y, dentro, busca el enlace de la noticia (un <a> cuyo href sea /politica/algo/), el titulo (clase 'titulo-teaser') y la fecha (clase 'fecha-teaser'). Ese es el mapeo etiqueta -> columna. Como el listado solo da titulo, url y fecha, el autor, resumen y tags se completan en la etapa 02.

7.2 Script 01 - recoleccion de links (HTML paginado)

Funcion parsear_fecha: dos formatos con regex

```
# 1) Patron maquina: AAAA-MM-DD [HH:MM(:SS)]
m = re.search(r"(\d{4})-(\d{2})-(\d{2})(?:[ T](\d{2}):(\d{2})(?::(\d{2}))?)?", texto)
# 2) Patron local: DD/MM/AAAA [- HH:MM]
m = re.search(r"(\d{2})/(\d{2})/(\d{4})(?:\s*-\s*(\d{2}):(\d{2}))?", texto)
```

La fecha en el HTML puede venir 'sucua' (con texto alrededor) y en dos formatos distintos: el de maquina (2026-07-16 14:17) o el local peruano (16/07/2026). parsear_fecha intenta primero un patron y luego el otro con expresiones regulares que capturan ano, mes, dia, hora

y minuto en grupos. Los (?:...)? marcan partes opcionales (la hora puede o no estar). Si ninguno coincide, devuelve None, y dentro_del_rango decide NO descartar la noticia por fecha cuando no se pudo parsear (para no perder noticias por un formato raro).

Descargar una pagina y leer las tarjetas

```
sufijo = "" if pagina == 1 else f"{pagina}/"
url_pagina = BASE_LISTADO.format(page=sufijo) # https://peru21.pe/politica/{page}
soup = BeautifulSoup(response.text, "html.parser")
for card in soup.select("article.node--type-article"):
    a = card.find("a", href=re.compile(r"^/politica/[a-z0-9-]+/?$"))
    if a is None: continue
    if re.match(r"^/politica/\d+/?$", a.get("href", "")):
        continue # es un enlace de paginacion, no una noticia
```

La pagina 1 es "" y el resto 'N/'. Cada noticia esta en un <article class='node--type-article'>. Dentro se busca el enlace cuyo href tenga forma /politica/<slug>/ (letras, numeros y guiones). Se descartan explícitamente los enlaces /politica/NUMERO/ porque esos son la paginacion, no noticias. select(...) usa selector CSS; find(..., href=regex) filtra por el atributo href con una expresion regular.

Título y fecha de cada tarjeta

```
titulo_el = card.find(class_=re.compile(r"titulo-teaser"))
titulo = titulo_el.get_text(strip=True) if titulo_el else a.get_text(strip=True)
fecha_el = card.find(class_=re.compile(r"fecha-teaser"))
if fecha_el: fecha = fecha_el.get_text(strip=True)
else:
    time_el = card.find("time")
    fecha = time_el.get("datetime", "") if time_el else ""
```

El titulo esta en un elemento con clase 'titulo-teaser' (si no, se usa el texto del propio enlace). La fecha en un elemento 'fecha-teaser', con respaldo al atributo datetime de un <time>. El listado solo entrega titulo, url y fecha; autor/resumen/tags se dejan vacios y los completa la etapa 02. El bucle avanza de pagina hasta que una pagina no trae noticias, o aparecen noticias fuera de los 2 anos, con una salvaguarda extra: si dos paginas seguidas no aportan URLs nuevas, tambien corta.

7.3 Script 02 - extraccion de contenido

Peru21 no trae autor/resumen/tags en el listado, asi que este 02 tambien los completa desde el HTML del articulo (ademas del cuerpo). Es reanudable y tiene un fallback de tags.

Autor, resumen y tags (con fallback)

```
# autor: primer enlace a /autor/
for a in soup.find_all("a", href=re.compile(r"/autor/")):
    t = re.sub(r"\s+", " ", a.get_text(strip=True)).strip()
    if t: autor = t; break
# resumen: meta description
meta = soup.find("meta", attrs={"name":"description"})
resumen = meta["content"].strip() if meta and meta.get("content") else ""
# tags: cada uno es un <div class="item-tags-full">
for bloque in soup.find_all(class_=re.compile(r"item-tags-full", re.I)):
    ...
if not tags: tags = _tags_desde_keywords(soup) # fallback
```

El autor sale del primer enlace a /autor/. El resumen, de la etiqueta <meta name='description'> del HTML (la que usan buscadores y redes). Los tags, de unos <div class='item-tags-full'>. La funcion auxiliar _tags_desde_keywords es un PLAN B: si el articulo no trae ese bloque de tags, usa las palabras clave que el sitio publica en <meta name='keywords'> y, si tampoco, en el JSON-LD (un bloque <script type='application/ld+json'> con metadatos estructurados). Asi casi nunca se quedan sin tags.

Elegir el contenedor con mas parrafos

```
cuerpos = soup.find_all("div", class_=re.compile(r"field--name-body"))
if cuerpos:
    contenedor = max(cuerpos, key=lambda b: len(b.find_all("p")))
```

El cuerpo esta en <div class='field--name-body'>, pero puede haber VARIOS (el de la nota mas los de notas relacionadas incrustadas). Se elige el que tenga MAS parrafos con max(..., key=len de sus <p>), que es casi siempre el cuerpo real. Esto evita traer texto de las notas relacionadas. Despues se hace el decompose de firmas/fechas/tags/embeds/relacionados y se extrae el texto de p/h2/h3/li como en los demas.

8. RPP Noticias

RPP no tiene API de listados, pero si un 'archivo' por fecha: la URL

<https://rpp.pe/archivo/politica/AAAA-MM-DD> muestra las noticias de politica de ese dia.

La estrategia es recorrer dia por dia hacia atras, desde hoy, hasta cubrir los 2 anos.

8.1 Como se construye la peticion (ejemplo real)

Como en Peru21, no hay query string: la 'consulta' es la URL, y lo que cambia es la FECHA dentro de la ruta. Se arma poniendo la fecha en formato AAAA-MM-DD:

```
Hoy:      GET https://rpp.pe/archivo/politica/2026-07-24
Ayer:     GET https://rpp.pe/archivo/politica/2026-07-23
Anteayer: GET https://rpp.pe/archivo/politica/2026-07-22 ... hacia atras
```

La respuesta es HTML. Cada noticia del dia viene en una tarjeta asi (simplificada):

```
<article class="news">
  <h2 class="news__title">
    <a href="/politica/...-noticia-1552634">Titulo de la noticia</a>
  </h2>
</article>
```

El script recorre cada <article class='news'>, toma el enlace dentro de <h2 class='news__title'> y se queda con su texto (titulo) y su href (url). El patron '-noticia-' seguido de numeros confirma que es una noticia y no otro tipo de enlace. La fecha no se saca del HTML: es la misma fecha del dia que se esta recorriendo. Autor, resumen y tags se completan en la etapa 02.

8.2 Script 01 - recoleccion de links (archivo por dia)

Funcion obtener_noticias_dia

```
BASE_ARCHIVO = "https://rpp.pe/archivo/politica/{fecha}" # fecha = AAAA-MM-DD
def obtener_noticias_dia(fecha):
    url_dia = BASE_ARCHIVO.format(fecha=fecha.strftime("%Y-%m-%d"))
    response = session.get(url_dia, headers=HEADERS, timeout=20)
    if response.status_code != 200: return []
    soup = BeautifulSoup(response.text, "html.parser")
    for art in soup.select("article.news"):
        a = art.select_one("h2.news__title a")
        if a is None: a = art.find("a", href=re.compile(r"-noticia-\d+"))
        if a is None: continue
        href = a.get("href", "")
        if "-noticia-" not in href: continue
        url_completa = href if href.startswith("http") else "https://rpp.pe" + href
        resultados.append((a.get_text(strip=True), url_completa))
```

Recibe una fecha, arma la URL del archivo de ese dia y descarga el HTML. Cada noticia

esta en un `<article class='news'>`; el titulo enlazado esta en `<h2 class='news__title'>` a. Si no se encuentra asi, hay un respaldo: cualquier enlace cuyo href tenga el patron `'-noticia-'` seguido de numeros (que es como RPP identifica sus notas). Se filtran los que no tengan `'-noticia-'` y se arma la url absoluta. Devuelve pares (titulo, url).

Bucle: recorrer dia por dia

```

hoy = date.today()
fecha_limite = hoy - timedelta(days=ANIOS_MAXIMOS * 365)
fecha_actual = hoy
while fecha_actual >= fecha_limite:
    encontrados = obtener_noticias_dia(fecha_actual)
    for titulo, url in encontrados:
        if url in urls_vistas: continue
        urls_vistas.add(url)
        noticias.append({"medio": "RPP", "fecha": fecha_actual.strftime("%Y-%m-%d"),
                        "titulo": titulo, "autor": "", "url": url,
                        "resumen": "", "tags": ""})
    fecha_actual -= timedelta(days=1)
    time.sleep(PAUSA_ENTRE_DIAS)

```

A diferencia de los demas, aqui el corte por fecha es el PROPIO bucle: se empieza en hoy y se resta un dia (`timedelta(days=1)`) en cada vuelta hasta llegar a la fecha limite (hace 2 anos). `urls_vistas` (un conjunto) evita duplicados si una noticia aparece en el archivo de mas de un dia. El listado solo da titulo, url y la fecha del dia; autor/resumen/tags se completan en la etapa 02.

8.3 Script 02 - extraccion de contenido

Es el script 02 mas completo, y sirvio de modelo para Peru21 y Canal N. Tiene helpers, fallback de tags, filtro de firma, y es reanudable con guardado cada 20.

Metadatos: autor, resumen, tags

```

# autor: primer enlace a /autor/; respaldo por clase article__author-name
for a in soup.find_all("a", href=re.compile(r"/autor/")):
    if a.get_text(strip=True): autor = a.get_text(strip=True); break
# resumen: meta description
# tags: enlaces dentro del bloque article__block--tags; si no, _tags_desde_keywords()

```

Igual que Peru21: autor del primer enlace `/autor/` (con respaldo por una clase de nombre de autor), resumen del meta description, y tags del bloque propio de etiquetas con el mismo plan B por keywords/JSON-LD.

Contenedor y limpieza

```

contenedor = soup.find("div", class_="article__content")
if not contenedor: contenedor = soup.find("article", class_="article")
if not contenedor: contenedor = soup.find("article")
# quitar: <article> relacionados incrustados, embeds, modulos gray-media,
#         y por clase news__|related|mas-leido|recirc|paginador|tags
for tarjeta in contenedor.find_all("article"): tarjeta.decompose()
for embed in
contenedor.find_all(["blockquote","iframe","script","style","figure","aside"]):
    embed.decompose()

```

El cuerpo esta en <div class='article__content'> (respaldos a <article>). Se eliminan las tarjetas de noticias relacionadas (que RPP incrusta como <article> dentro del cuerpo), los embeds, los modulos de video/podcast (clase 'gray-media', que dejan titulos tipo 'EP232 | INFORMES'), y restos de modulos por clase (relacionados, mas leido, recirculacion, paginador, tags). Tambien se quita la bajada (primer <h2> sin clase), que ya se guardo como resumen.

Extraccion con filtro de firma

```

for el in contenedor.find_all(["p","h2","h3","li"]):
    texto = _texto_limpio(el)
    if not texto: continue
    # descartar la firma ("por Redaccion RPP"): <p> corto que empieza con "por"
    if el.name == "p" and len(texto) < 60 and re.match(r"(?i)^por\s+", texto): continue
    if any(frase in texto.upper() for frase in FRASES_RUIDO): continue
    if el.name == "li": texto = f"- {texto}"
    texto_limpio.append(texto)

```

Ademas del filtro comun de frases de ruido, RPP descarta la firma del autor cuando aparece como un parrafo corto (menos de 60 caracteres) que empieza con 'por' (la expresion regular '(?i)^por\s+' ignora mayusculas). Asi 'por Redaccion RPP' no se cuela dentro del cuerpo.

9. Canal N

Canal N es una aplicacion 'Next.js'. No expone ningun listado historico: su feed de seccion se corta en unas 50 noticias recientes, no hay archivo por fecha ni sitemap profundo. Pero se descubrieron tres cosas que hacen posible el scraping, y que llevan a un enfoque distinto: un solo script.

9.1 Los tres hallazgos

- Cada noticia tiene un ID numerico correlativo al final de su URL (.../-n493589). Como son consecutivos, se pueden recorrer hacia atras (del mas nuevo al mas viejo).
- Se puede abrir cualquier noticia por su ID con un slug inventado: la URL `https://canaln.pe/actualidad/x-n<ID>` redirige sola a la direccion real. Asi no hace falta conocer el titulo/slug exacto de cada noticia.
- Cada articulo trae TODOS sus datos en un JSON incrustado en el HTML, dentro de `<script id='__NEXT_DATA__'>`. De ahi salen titulo, fecha, resumen, tags, categoria y el cuerpo, sin adivinar selectores de HTML.

Por eso Canal N usa UN solo script (recorrer por ID y extraer el JSON en el mismo paso), en vez del pipeline de dos etapas. El resto (esquema de 8 columnas, sesion robusta, corte a 2 anos, CSV) es igual que los demas.

9.2 Como se construye la peticion (ejemplo real)

Canal N no tiene un listado que pedir, asi que la 'peticion' es distinta: se abre una noticia por su ID numerico, con un slug (nombre) cualquiera, y el sitio redirige solo a la direccion correcta. Los IDs son correlativos, asi que se van pidiendo de mayor a menor:

```
GET https://canaln.pe/actualidad/x-n493589 -> redirige a la noticia n493589
GET https://canaln.pe/actualidad/x-n493588 -> redirige a la noticia n493588
GET https://canaln.pe/actualidad/x-n493587 ... y asi hacia atras
```

Lo que devuelve es la pagina HTML, pero con un truco: al ser una app Next.js, TODOS los datos de la noticia estan en un JSON escondido dentro del HTML, en una etiqueta `<script id="__NEXT_DATA__">`. Se extrae ese JSON y adentro, en `props.pageProps.compact`, esta todo (version simplificada):

```
{
  "pubtime": 1784685614,      -> fecha (epoch en segundos)
  "titulo": "Titulo de la nota", -> titulo
```

```

"bajada": "Resumen ...",          -> resumen
"categoria": {"url": "actualidad"}, -> para filtrar solo Actualidad
"tags": [{"name": "Vizcarra"}],    -> tags
"bloques": [                      -> el cuerpo, partido en bloques
    {"tipo": "texto", "data": "<p>Primer parrafo...</p>"},
    {"tipo": "imagen", "data": "..."},          (se ignora)
    {"tipo": "texto", "data": "<p>Segundo parrafo...</p>"}
]
}

```

Así, con UNA sola petición por noticia se obtiene todo (por eso Canal N no necesita dos etapas). El cuerpo se arma juntando los bloques de tipo 'texto'; el autor sale del HTML (un enlace /autor/). Si el ID no existe, el sitio responde 404 y se salta.

9.3 La función limpiar_html

```

def limpiar_html(texto):
    plano = BeautifulSoup(htmllib.unescape(str(texto)), "html.parser").get_text(" ",
strip=True)
    return re.sub(r"\s+", " ", plano).strip()

```

El título, el resumen y los tags llegan como HTML con entidades (ej. '´'). Esta función, que se reutiliza en varios campos: (1) htmllib.unescape convierte las entidades ('´' -> 'ó'); (2) BeautifulSoup quita las etiquetas y deja solo texto; (3) re.sub colapsa espacios. Tenerla en un solo lugar evita repetir esa limpieza cinco veces.

9.4 La función obtener_notas: el corazón

Recibe un ID y devuelve una de tres cosas, lo que mantiene el bucle principal muy simple:

- un diccionario con la fila lista, si la noticia es de la categoría Actualidad;
- la palabra "DETENER", si la noticia ya supera los 2 años;
- None, si ese ID no sirve (no existe, es de otra categoría, o hubo error).

Descargar por ID y leer el JSON

```

r = session.get(f"https://canaln.pe/actualidad/n-n{id_notas}", headers=HEADERS,
timeout=20)
if r.status_code != 200:          # 404 = ese ID no existe (hueco); se salta
    return None
m = re.search(r'<script id="__NEXT_DATA__" type="application/json">(.*?)</script>',
r.text, re.S)
compacto = json.loads(m.group(1))["props"]["pageProps"].get("compacto")

```

Pide la URL por ID (que redirige a la real). Un 404 significa que ese ID no corresponde a ninguna noticia (hay huecos: IDs borrados o de otros tipos de contenido); se ignora y se sigue. Con una expresión regular se captura el JSON del <script id='__NEXT_DATA__'>

(re.S hace que el '.' tambien tome saltos de linea, porque el JSON ocupa varias lineas), se parsea y se baja hasta props.pageProps.compacto, el objeto con los datos de la nota.

Corte por fecha y filtro de categoria

```
fecha = datetime.fromtimestamp(int(compacto["pubtime"]),
tz=timezone.utc).strftime("%Y-%m-%d")
if fecha and fecha < fecha_limite:
    return "DETENER"
if (compacto.get("categoria") or {}).get("url") != "actualidad":
    return None
```

pubtime es la fecha en epoch (segundos); se convierte a 'AAAA-MM-DD'. Como los IDs van en orden cronologico, en cuanto una noticia (de cualquier categoria) cae bajo la fecha limite se devuelve 'DETENER' para cortar todo el recorrido. Por eso la fecha se revisa ANTES de filtrar por categoria: sirve de marcador cronologico general. Luego, solo interesan las de categoria 'actualidad'; el resto (Peru, Deportes, Internacionales) se descarta con None.

Autor, tags y cuerpo (bloques)

```
enlace_autor = soup.find("a", href=re.compile(r"/autor/"))
autor = re.sub(r"[\s...]+$", "", limpiar_html(enlace_autor.get_text(" ", strip=True)))
tags = ", ".join(dict.fromkeys(limpiar_html(t["name"]) for t in compacto.get("tags") or []))
lineas = []
for bloque in compacto.get("bloques") or []:
    if bloque.get("tipo") not in ("texto", "subtitulo", "sumario", "cita", "lista", "html"):
        continue
    bloque_soup = BeautifulSoup(htmllib.unescape(bloque["data"]), "html.parser")
    for basura in bloque_soup.find_all(["figure", "iframe", "script", ...]):
        basura.decompose()
    for elem in bloque_soup.find_all(["p", "h2", "h3", "li"]):
        texto = limpiar_html(elem.get_text(" ", strip=True))
        if not texto or any(f in texto.upper() for f in FRASES_RUIDO): continue
        lineas.append(f"- {texto}" if elem.name == "li" else texto)
```

El autor sale del enlace /autor/ (con un re.sub que quita simbolos decorativos al final del nombre). El cuerpo viene partido en 'bloques' dentro del JSON; se toman solo los de texto (ignorando imagenes/video), cada uno es a su vez un fragmento de HTML que se limpia con decompose y del que se extraen parrafos. Se filtran vacios y frases de ruido. Finalmente se arma el diccionario de 8 columnas y se devuelve.

9.5 Reanudar y el bucle principal

```
if os.path.exists(ARCHIVO_CHECKPOINT):
    nid = int(open(ARCHIVO_CHECKPOINT).read().strip())    # continuar desde el
checkpoint
else:
    sitemap = session.get("https://canaln.pe/sitemap/news", ...).text
    nid = max(int(x) for x in re.findall(r"-n(\d+)", sitemap))    # ID mas reciente
while nid > 0 and procesados < 60000:
    nota = obtener_nota(nid)
    time.sleep(0.15)
    if nota == "DETENER": break
    if isinstance(nota, dict) and nota["url"] not in urls_vistas:
        urls_vistas.add(nota["url"]); filas.append(nota)
    procesados += 1; nid -= 1
    if procesados % 50 == 0:
        ...guardar CSV y escribir el checkpoint con el proximo ID...
```

El punto de partida es el ID del checkpoint si existe; si es la primera vez, no hay datos previos, así que se le pregunta al sitio cual es el ID mas reciente: se descarga el sitemap de noticias y se toma el mayor ID (regex '-n(\d+)'). Desde ahí el bucle baja de ID en ID. Toda la lógica pesada está en `obtener_nota`, así que el bucle queda claro: pide la nota, espera, corta si es 'DETENER', o la guarda si es un diccionario nuevo. Cada 50 IDs guarda el CSV y el checkpoint (el próximo ID), de modo que si se corta, la siguiente ejecución continúa justo donde iba. El tope de 60000 es solo una red de seguridad contra bucles infinitos.

10. Union y normalizacion de los datasets

Cada medio deja su propio CSV. Estos dos scripts, que viven en la carpeta raiz del proyecto, juntan todo en un unico archivo y arreglan el orden de columnas. Son el 'ultimo paso' del flujo, y el mas relevante para un curso de base de datos, porque producen la tabla final.

10.1 unir_noticias_completas.py

Junta los '*_noticias_completas.csv' de todos los medios en un solo archivo, noticias_completas_unificado.csv, dejando el esquema comun de 8 columnas y sin duplicados.

Buscar los archivos automaticamente (glob)

```
import glob
patron = os.path.join(BASE_DIR, "*", "*_noticias_completas.csv")
rutas = sorted(glob.glob(patron))
```

glob.glob busca archivos que encajen con un 'patron' que admite comodines: el '*' significa 'cualquier cosa'. Aqui el patron es 'CUALQUIER_CARPETA / CUALQUIER_NOMBRE terminado en _noticias_completas.csv'. Asi encuentra el CSV final de cada medio sin tener que listarlos a mano; si maniana agregas otro medio, lo tomara solo. sorted() los ordena alfabeticamente para que el resultado sea estable.

Normalizar cada CSV al esquema (reindex)

```
def cargar_normalizado(ruta):
    df = pd.read_csv(ruta, encoding="utf-8-sig", dtype=str)
    extra = [c for c in df.columns if c not in COLUMNAS]
    if extra: print(f"    (columnas ignoradas: {extra})")
    df = df.reindex(columns=COLUMNAS) # deja exactamente COLUMNAS, en ese orden
    return df
```

Lee el CSV forzando dtype=str: asi TODAS las celdas se leen como texto y las vacias no se convierten en el numero especial NaN ni en el texto 'nan' (un problema tipico de pandas).

reindex(columns=COLUMNAS) es la pieza clave: deja el DataFrame con

EXACTAMENTE las 8 columnas del esquema y en ese orden; si a un medio le falta alguna (p. ej. El Comercio a veces sin 'tags'), la crea vacia, y si le sobra alguna, la descarta (avisando cuales).

Concatenar, limpiar y deduplicar

```
unido = pd.concat(partes, ignore_index=True) # apila todas las tablas
unido = unico.fillna("")                    # NaN -> cadena vacia
for col in COLUMNAS:
```

```

unido[col] = unico[col].astype(str).str.strip() # quita espacios sobrantes

con_url = unico["url"] != ""
dup = unico[con_url].duplicated(subset="url", keep="first")
if dup.any():
    idx_dup = unico[con_url].index[dup]
    unico = unico.drop(index=idx_dup).reset_index(drop=True)

```

pd.concat apila las tablas de todos los medios una debajo de otra (ignore_index=True vuelve a numerar las filas desde 0). fillna("") convierte cualquier NaN restante en texto vacío, y .str.strip() recorta espacios al inicio/fin de cada celda. Luego se DEDUPLICA por 'url': duplicated(subset='url', keep='first') marca como repetidas las filas cuya url ya apareció antes (conservando la primera). Se hace solo sobre las filas con url no vacía, para no borrar por error las que no tienen url. La url funciona como identificador único de cada noticia. Finalmente se guarda el resultado con to_csv(..., encoding='utf-8-sig') y se imprime un resumen: cuántas filas había, cuántos duplicados se quitaron, y cuántas noticias quedaron por medio. El bloque 'if __name__ == "__main__": main()' del final significa 'ejecuta main() solo si corro este archivo directamente' (una convención estándar de Python).

10.2 normalizar_orden_csv.py

Es una utilidad opcional: reordena las columnas de los CSV ya generados al esquema estándar SIN volver a scrapear. Sirve si un archivo quedó con las columnas en otro orden.

```

COLS_POLITICA = ["medio", "fecha", "titulo", "autor", "url", "resumen", "tags"]
COLS_COMPLETAS = COLS_POLITICA + ["texto_completo"]

def normalizar(ruta, columnas):
    df = pd.read_csv(ruta, encoding="utf-8-sig", dtype=str)
    df = df.reindex(columns=columnas) # reordena / crea faltantes / descarta
    sobrantes
    df = df.fillna("")
    df.to_csv(ruta, index=False, encoding="utf-8-sig") # reescribe el MISMO archivo

```

Maneja dos esquemas: los '*_politica.csv' (7 columnas, sin texto) y los '*_noticias_completas.csv' (8 columnas). Por cada archivo aplica el mismo truco del reindex y lo reescribe en su sitio. Es 'idempotente': si el archivo ya estaba bien, el contenido no cambia; si estaba desordenado, lo deja en el orden correcto. Solo toca el ORDEN de las columnas, nunca los datos. Informa por consola si cada archivo quedó 'OK (sin cambios)' o 'REORDENADO'.

11. Como ejecutar el proyecto

Guia practica para correr todo desde cero y obtener el dataset final.

11.1 Preparar el entorno

```
# 1) crear un entorno virtual (aisla las librerias del proyecto)
python -m venv venv
# 2) activarlo
venv\Scripts\activate      # en Windows
# 3) instalar las dependencias
pip install -r requirements.txt
```

Un 'entorno virtual' (venv) es una copia aislada de Python donde se instalan las librerias del proyecto (requests, BeautifulSoup4, pandas...) sin afectar al resto del sistema.

requirements.txt es la lista de esas librerias; pip las instala todas de una vez.

11.2 Ejecutar cada scraper

Para los cinco medios con dos etapas, se corre primero el 01 (recolectar links) y luego el 02 (extraer el texto). Ejemplo con RPP:

```
cd RPP
python 01_recoleccion_links_rpp.py      # crea rpp_politica.csv
python 02_extraccion_contenido_rpp.py   # crea rpp_noticias_completas.csv
```

Canal N es de una sola etapa:

```
cd "Canal N"
python scraping_canalN.py                # crea canalN_noticias_completas.csv
```

- Los scripts 02 y el de Canal N son REANUDABLES: si se cortan (corte de luz, internet), se vuelve a correr el mismo comando y continua donde iba, sin repetir lo ya hecho.
- Cuanto recolectar se controla con la constante ANIOS_MAXIMOS al inicio de los scripts 01 (y de Canal N): por defecto 2 años. Bajarla sirve para hacer pruebas rapidas.
- Cada petición lleva una pequeña pausa (time.sleep) para no saturar los servidores; por eso una corrida completa de la etapa 02 puede tardar bastante (miles de noticias).

11.3 Unir todo en el dataset final

```
# desde la carpeta raiz del proyecto
python unir_noticias_completas.py      # crea noticias_completas_unificado.csv
python normalizar_orden_csv.py         # (opcional) reordena columnas si hiciera falta
```

unir_noticias_completas.py recoge automaticamente el CSV de cada medio y produce el

archivo unico noticias_completas_unificado.csv, ya deduplicado. Ese es el entregable de datos final.

12. El modelo de datos (esquema de salida)

Todos los scrapers producen la misma tabla. Hay dos 'niveles' de archivo y el unificado:

- *_politica.csv: salida de la etapa 01. 7 columnas (sin texto_completo).
- *_noticias_completas.csv: salida de la etapa 02. Las 8 columnas completas.
- noticias_completas_unificado.csv: todos los medios juntos, deduplicado.

12.1 Las 8 columnas, en detalle

medio - nombre del diario, como texto fijo. Ej: 'El Comercio', 'RPP', 'Canal N'. Identifica de que fuente viene cada noticia.

fecha - fecha de publicacion. El formato exacto varia segun lo que entrega cada medio: ISO 8601 en El Comercio (2026-05-11T17:25:59Z), 'AAAA-MM-DD HH:MM:SS' en La Republica y El Peruano, y 'AAAA-MM-DD' en RPP y Canal N. Todas quedan comparables/ordenables.

titulo - el titular de la noticia (texto).

autor - la firma. Puede quedar VACIO cuando el medio no expone autor (El Peruano en su API) o cuando la nota no lo indica.

url - la direccion completa de la noticia. Es el campo mas importante como IDENTIFICADOR: es unico por noticia y por eso se usa para eliminar duplicados al unir.

resumen - la 'bajada' o subtítulo (texto breve que resume la nota).

tags - las etiquetas/temas, como texto separado por comas. Ej: 'Congreso, Elecciones 2026, JNE'. Un mismo campo guarda varias etiquetas.

texto_completo - el cuerpo entero de la noticia en una sola celda, ya limpio de publicidad y recomendados.

12.2 Como se llevaria a una base de datos

Para el curso de Base de Datos, este CSV se puede cargar en una tabla. Una definicion posible:

```
CREATE TABLE noticia (
  id          SERIAL PRIMARY KEY,
  medio       VARCHAR(50) NOT NULL,
  fecha       TIMESTAMP,
  titulo      TEXT        NOT NULL,
  autor       VARCHAR(200),
```

```

url          TEXT          NOT NULL UNIQUE,  -- identificador natural
resumen      TEXT,
texto_completo TEXT
);

```

La 'url' se marca UNIQUE porque no debe repetirse (es justo lo que el script ya garantiza al deduplicar). La columna 'tags' merece atencion aparte: guardar varias etiquetas en un solo campo separado por comas ROMPE la primera forma normal (1FN), que pide un valor por celda. Lo correcto en una base relacional es separarlas en su propia tabla y relacionarlas:

```

CREATE TABLE tag (
    id      SERIAL PRIMARY KEY,
    nombre  VARCHAR(100) UNIQUE
);
CREATE TABLE noticia_tag (      -- relacion muchos-a-muchos
    noticia_id INT REFERENCES noticia(id),
    tag_id     INT REFERENCES tag(id),
    PRIMARY KEY (noticia_id, tag_id)
);

```

Asi cada noticia puede tener muchas etiquetas y cada etiqueta pertenecer a muchas noticias, sin repetir texto. Al cargar el CSV, se separaria la columna 'tags' por comas y se llenaria la tabla intermedia. Este es el paso natural entre el scraping (que produce el CSV plano) y el modelado relacional del curso.

13. Comparativa de estrategias

La diferencia esencial entre los seis esta en como consiguen la lista de noticias. El resto (esquema de 8 columnas, limpieza del cuerpo, corte a 2 anos, CSV en utf-8-sig) es comun.

El Comercio

Tecnica: API J

El Peruano

Tecnica: API J

La Republica

Tecnica: API J

Peru21

Tecnica: HTM

RPP

Tecnica: Arch

Canal N

Tecnica: IDs c

En cuanto a completar autor/resumen/tags: El Comercio y La Republica los obtienen ya en la etapa 01 (su API los trae); El Peruano, Peru21 y RPP los completan en la etapa 02 desde el HTML del articulo; Canal N los saca del JSON en su unico paso. Los cinco medios con pipeline de dos etapas producen dos CSV (uno de links y uno completo); Canal N produce directamente el CSV completo.

14. Glosario de terminos

Mini-diccionario de las palabras tecnicas que aparecen en el documento, en lenguaje llano.

- Web scraping: extraer datos de paginas web de forma automatica con un programa.
- Peticion / request: mensaje que un programa envia a un servidor para pedirle algo.
- GET: el tipo de peticion que sirve para 'obtener' (pedir) contenido de una URL.
- Status code (codigo de estado): numero que indica como fue la peticion (200 = OK, 404 = no existe, 403 = prohibido, 500 = error del servidor).
- URL: la direccion de un recurso en la web (https://dominio/ruta?parametros).
- Endpoint: una URL de una API pensada para entregar datos a programas, no una pagina.
- API: 'puerta de datos' de un sitio; una direccion que devuelve datos crudos (normalmente JSON) en vez de una pagina para ver.
- Query string / parametros: la parte de la URL despues de '?', con filtros clave=valor separados por '&' (ej. ?page=2&limit=24). Es 'la query'.
- Header / cabecera: dato extra que acompaña la peticion (quien pide, desde donde, etc.).
- User-Agent: cabecera que identifica el navegador; se falsea para no ser bloqueado.
- Referer: cabecera que indica desde que pagina se hace la peticion (La Republica la exige).
- JSON: formato de datos estructurado con objetos {clave: valor} y listas [...].
- HTML: el lenguaje de las paginas web, hecho de etiquetas anidadas.
- Etiqueta / tag: cada elemento del HTML (ej. <p>, <a>, <article>).
- Atributo: informacion dentro de una etiqueta (ej. href, class, id).
- class / id: atributos que el sitio usa para dar estilo; sirven para localizar elementos.
- Parsear: analizar un texto (HTML o JSON) para convertirlo en algo que el programa puede recorrer y del que puede sacar datos.
- Selector: forma de senalar que elemento del HTML se quiere (por etiqueta, clase o id).
- BeautifulSoup: libreria de Python que parsea HTML y permite buscar elementos.
- requests: libreria de Python que hace las peticiones a la web.
- pandas / DataFrame: libreria y estructura de tabla (filas y columnas) en memoria.

- CSV: archivo de texto con datos en tabla, separados por comas; se abre en Excel.
- Epoch: forma de guardar una fecha como un numero de segundos (o milisegundos) desde el 1 de enero de 1970.
- Regex (expresion regular): patron para buscar/extraer texto (ej. sacar el ID de una URL).
- Paginacion: dividir una lista larga en paginas; el scraper las recorre una por una.
- Slug: la parte legible de una URL que identifica el articulo (ej. /politica/mi-noticia).
- DevTools / Network: herramientas del navegador (tecla F12) para ver las peticiones y el HTML de un sitio; con ellas se descubre como scrapearlo.